

EVALUACIÓN 3 - MODELOS DE CLASIFICACIÓN

Detección de Fraude con Ajuste de Modelos y Threshold

Contexto de negocio

La empresa fintech **PaySecure** desea implementar un sistema de detección de fraude en tiempo real para sus transacciones.

El problema es crítico porque:

- Detectar un fraude tarde implica pérdidas económicas
- Detectar incorrectamente una transacción legítima afecta la experiencia del cliente
- Existe un **fuerte desbalance de clases**, donde la mayoría de las transacciones son legítimas

El objetivo es construir un modelo que permita **identificar transacciones fraudulentas** de forma eficiente.

Descripción del dataset

El dataset contiene información de transacciones.

Variables continuas

- transaction_amount → monto de la transacción
- account_balance → saldo disponible
- transaction_time_seconds → hora del día (en segundos)
- avg_transaction_amount_7d → promedio últimos 7 días
- std_transaction_amount_7d → variabilidad reciente

Variables discretas

- transactions_last_1h → número de transacciones en la última hora
- transactions_last_24h → número en últimas 24 horas
- failed_attempts → intentos fallidos previos
- num_devices_used → dispositivos distintos usados

Variables categóricas

- transaction_type → ("payment", "transfer", "withdrawal", "purchase")
- device_type → ("mobile", "desktop", "tablet")
- location_region → ("urban", "suburban", "rural")
- is_foreign_transaction → (0/1)
- is_high_risk_country → (0/1)

Objetivo de la evaluación

Desarrollar, optimizar y evaluar modelos de clasificación binaria para predecir:

- is_fraud = 1 que indica una transacción fraudulenta
- is_fraud = 0 que indica una transacción legítima

Parte 1: Preprocesamiento (20 puntos)

1. Identifique variables numéricas y categóricas
2. Realice el preprocesamiento necesario:
 - a. Encoding de variables categóricas
 - b. Escalamiento si corresponde
3. Divida los datos en entrenamiento y prueba
4. Justifique brevemente sus decisiones

Parte 2: Entrenamiento y optimización con GridSearchCV (30 puntos)

Entrene **DOS** modelos de clasificación usando los algoritmos revisados:

- Logistic Regression
- Decision Tree Classifier

Para cada modelo:

1. Defina una **grilla de hiperparámetros**
2. Aplique **GridSearchCV**
3. Indique:
 - a. Mejores hiper parámetros encontrados

- b. Métrica utilizada para optimización
4. Compare el rendimiento entre modelos
5. Justifique cuál modelo selecciona y por qué

Parte 3: Evaluación base del modelo (15 puntos)

Utilizando el mejor modelo:

1. Realice predicciones sobre el conjunto de prueba con **threshold = 0.5 (recuerde que este es el valor por defecto)**
2. Calcule:
 - a. Accuracy
 - b. Precision
 - c. Recall
 - d. F1-score
3. Obtenga la **matriz de confusión**
4. Interprete los resultados considerando el contexto de fraude

Parte 4: Ajuste de threshold (25 puntos)

El objetivo de esta sección es analizar cómo cambia el comportamiento del modelo al modificar el umbral de decisión.

4.1 Evaluación con distintos thresholds (umbrales)

Evalúe el modelo con los siguientes thresholds:

- 0.3
- 0.5
- 0.7

Para cada uno de los valores del umbral debe:

- Generar predicciones
- Calcular:
 - Precision
 - Recall
 - F1-score
- Obtener la **matriz de confusión**
- Generar un archivo CSV con las predicciones y probabilidades obtenidas; el nombre del archivo debe contener el threshold con el cual fueron obtenidas las predicciones.

4.2 Análisis comparativo

1. Compare los resultados entre thresholds
2. Analice:
 - a. ¿Qué ocurre con los falsos positivos?
 - b. ¿Qué ocurre con los falsos negativos?
 - c. ¿Cómo cambian precision y recall?

4.3 Decisión de negocio

Responda:

- ¿Qué threshold, de todos los analizados, elegiría para este problema?
- Justifique su respuesta considerando:
 - Riesgo de fraude
 - Impacto en clientes
 - Métricas obtenidas

Parte 5: Conclusiones (10 pts)

- ¿Qué modelo resultó mejor y por qué?
- ¿Qué variable(s) parecen más influyentes?
- ¿Qué mejoras propondría al sistema?

Requisitos técnicos

El desarrollo debe incluir:

- Uso de GridSearchCV
- Uso de probabilidades (predict_proba)
- Cálculo de métricas con sklearn
- Uso de matriz de confusión (confusion_matrix)

Penalizaciones transversales

Puedes aplicar descuentos adicionales:

SITUACIÓN	DESCUENTO
✗ No usar predict_proba en threshold	5 puntos
✗ Evaluar solo con accuracy	15 puntos
✗ Data leakage	10 puntos
✗ Código que no ejecuta	15 puntos
✗ Ausencia de CSV con predicciones y probabilidades	15 puntos

Indicadores de evaluación INDIVIDUAL

Cada estudiante deberá responder DOS PREGUNTAS y la calificación dependerá de la calidad de la respuesta entregada:

CALIDAD RESPUESTA	CALIFICACIÓN
BIEN	7,0
REGULAR	3,5
DEFICIENTE	1,5
NO RESPONDE O NO SABE	1,0

Calificación final

40% calificación grupal + 60% calificación individual

Formato de entrega

Enlace de repositorio Git que responda al siguiente esquema de carpetas (actualizado agregando el notebook de la entrega anterior)

En la carpeta ingestion debe quedar el set de datos de esta evaluación y el notebook DEBE CARGAR EL ARCHIVO CSV DESDE ESA CARPETA.

```
620454/
├── README.md # Incluye identificación de integrantes y descripción del repositorio
├── data/
│   ├── ingestion/      # Datos crudos
│   └── cleaned/        # Datos limpios y transformados
├── notebooks/
│   └── E1-Preparacion.ipynb      # Preparación de datos
```

Condiciones de entrega

- Plazo de entrega – jueves 11 de junio HASTA LAS 23:00 PM
- Una entrega por grupo en el enlace de ADECCA
- Preguntas – viernes 12 de junio al inicio de la clase
- Se considera el mismo repositorio entregado en la evaluación anterior